

Final Year Project Report: Context-Aware Marathi Sign Language Translator

1. Introduction

What the project is about

This project is an AI-powered Sign Language Recognition System designed specifically for **Marathi Sign Language (MSL)**. The system uses a standard computer webcam to track a user's hand gestures in real-time, translates those gestures into corresponding Marathi words, and uses a smart Natural Language Processing (NLP) pipeline to build grammatically correct sentences. Finally, the system speaks the translated sentence aloud using Text-to-Speech (TTS).

Problem Statement

Deaf and hard-of-hearing individuals face significant communication barriers when interacting with people who do not understand sign language. While many global sign languages (like American Sign Language) have extensive research and translation tools, regional languages like Marathi lack robust, automated translation systems.

Real-World Use Case

This system serves as an assistive communication tool. For instance, a deaf individual at a railway station can sign "Ticket" and "Pune", and the system will instantly translate and speak the sentence "मला पुणेचे तिकीट पाहिजे" (I want a ticket for Pune) aloud to the ticket clerk, facilitating seamless, independent communication.

2. Project Overview

How the system works (End-to-End Flow)

The system operates in a continuous, real-time pipeline:

- 1. Input:** The webcam captures a continuous video feed of the user.
- 2. Processing:**
 - Hand Tracking:** The system isolates the user's hand and extracts exactly 21 3D coordinate points (landmarks) representing the joints of the hand.
 - Normalization:** The coordinates are mathematically scaled and shifted so that the gesture looks identical to the AI whether the hand is close to the camera or far away.
 - Classification:** A Deep Learning neural network analyzes the normalized coordinates and classifies them into a specific gesture label (e.g., "g_ticket").
 - NLP & Context Matching:** The system detects the context (e.g., Transport, Shopping) based on the gestures, groups them, and fits them into grammatically

correct Marathi sentence templates.

3. **Output:** The graphical UI displays the final translated Marathi sentence, and a Text-to-Speech engine converts the text into audible speech.
-

3. Technologies Used

Programming Languages

- **Python (3.10):** The core language used for the entire application due to its strong AI and computer vision ecosystem.

Frameworks and Libraries

- **OpenCV (cv2):** Used for capturing webcam video, processing video frames, and rendering live visual feedback (like the skeletal overlay).
 - **MediaPipe:** Google's open-source framework used specifically for high-fidelity hand tracking and 3D landmark extraction.
 - **TensorFlow / Keras:** Used to build, train, and run the Deep Learning neural network (classifier).
 - **NumPy:** Handled heavy mathematical operations, particularly array manipulations and coordinate normalization.
 - **CustomTkinter:** Used to build the sleek, modern graphical user interface (GUI).
 - **pyttsx3 / gTTS:** Used for the Text-to-Speech audio output.
 - **Scikit-Learn & Seaborn:** Used to evaluate model accuracy and generate confusion matrices.
-

4. Why These Technologies Were Chosen

Justification and Advantages

- **MediaPipe over Custom CNNs:** Originally, one might train a Convolutional Neural Network (CNN) on raw images of hands. However, raw images are heavily affected by lighting, background noise, and skin color. MediaPipe was chosen because it ignores the visual background and extracts pure mathematical skeletons (landmarks). This makes our AI vastly more robust and lightweight.
 - **Deep Neural Network (DNN) over SVM/KNN:** A standard multi-layer perceptron (DNN) was chosen to classify the landmarks. Since the input data is extremely structured (63 numerical values representing X,Y,Z coordinates), a lightweight DNN trains in seconds and predicts in milliseconds, allowing for real-time video processing.
 - **CustomTkinter over Tkinter/PyQt:** CustomTkinter was chosen for the UI because it provides a highly polished, modern, dark-mode aesthetic out-of-the-box without the heavy learning curve and deployment overhead of PyQt.
-

5. Gesture Detection Process

How Gestures are Captured & Processed

1. **Extraction:** As the user moves, OpenCV passes each video frame to MediaPipe. MediaPipe scans the image and identifies 21 keypoints (knuckles, fingertips, wrist) on the hand.
 2. **Coordinate Normalization:** This was a critical step in the project. Raw coordinates represent pixel locations on the screen. If a user moves their hand to the left side of the screen, the raw numbers change drastically, confusing the AI.
 - We implemented an algorithm that makes the **wrist (Landmark 0) the origin (0,0,0)**.
 - All other 20 points are calculated relative to the wrist.
 - The coordinates are then divided by the maximum distance across the hand, effectively scaling every hand to the exact same size.
 - *Result:* The AI recognizes the gesture perfectly regardless of where the hand is on the screen or how far away the user is standing.
-

6. Data Collection & Annotation

Dataset Creation

To train the model, we developed a custom data collection script (`collect_data.py`). The script displays a live camera feed with a skeletal overlay, prompting the user to perform specific gestures (e.g., "One", "Two", "Buy", "Ticket", "Mobile"). For each gesture, the system automatically captures 300 frames of normalized landmark data.

Annotation Process

The system utilizes a unified `labels.json` dictionary. As data is captured, it is automatically assigned an integer ID (0, 1, 2...) corresponding to its label. This guarantees that the labels used during training perfectly match the labels used during live webcam prediction.

Challenges Faced

Initially, data was collected without normalization. This resulted in a model that only recognized gestures if the hand was held in the exact same spot it was trained in. We solved this by scrapping the raw dataset, implementing the normalization math, and re-collecting the data.

7. Model Details

Architecture

The brain of the gesture recognition is a Deep Neural Network built with TensorFlow/Keras.

- **Input Layer:** 63 neurons (flattened array of 21 landmarks $\times$ 3 coordinates [X, Y, Z]).
- **Hidden Layer 1:** 128 neurons with ReLU (Rectified Linear Unit) activation.
- **Hidden Layer 2:** 64 neurons with ReLU activation.
- **Output Layer:** A Softmax layer with neurons equal to the number of trained gestures (11).

Training Process & Hyperparameters

- **Optimizer:** Adam (Adaptive Moment Estimation) for efficient gradient descent.
 - **Loss Function:** Sparse Categorical Crossentropy.
 - **Epochs:** 50
 - **Batch Size:** 16
 - The training process splits the dataset, using 80% of the frames to learn and 20% to validate its learning.
-

8. Performance Metrics

Calculation

To evaluate the model, we used a subset of unseen data to calculate precision (how many predicted gestures were actually correct) and recall (how many actual gestures were successfully found).

Confusion Matrix

Below is the Confusion Matrix generated after our final training phase. It maps the *Actual Gestures* against the *Predicted Gestures*. A perfect model has a solid diagonal line of dark squares.

![Confusion Matrix](file:///E:/Projects/Sign-3.0/models/confusion_matrix.png)

9. Model Accuracy Analysis

Final Accuracy Achieved

The model achieved a phenomenal **100% Test Accuracy (1.00 F1-Score)**. *See the exact training progression in the graphs below:*

Accuracy Graph ![Accuracy](file:///E:/Projects/Sign-3.0/models/accuracy.png)

Loss Graph ![Loss](file:///E:/Projects/Sign-3.0/models/loss.png)

Reasons for High Accuracy

This level of accuracy was achieved because we removed the "noise" of raw images. By feeding the AI strictly normalized, mathematical skeletal data, the AI only had to learn the geometric shapes of the hand. Because the dataset was collected cleanly, the geometric shapes were highly distinct, leading to 0% misclassification.

10. How Gestures Convert to Words (NLP Pipeline)

Mapping Logic

Translating a single gesture to a single word is easy, but humans speak in sentences. We built

a dynamic ContextManager and SentenceBuilder.

1. **Context Auto-Detection:** If the user signs "Mobile", the system intelligently locks into the **Shopping** context. If they sign "Ticket", it locks into **Transport**.
 2. **Grammar Templates:** Gestures are assigned types (quantity, item, query_price).
 3. **Template Filling:** If the user signs "Ticket" (item) and "Two" (quantity) and "Buy" (action), the system ignores the order they were signed in, finds the matching template for the Transport context, and outputs the grammatically perfect Marathi string: मला दोन तिकीट पाहिजे. (I want two tickets).
-

11. Challenges & Solutions

1. Jittery Translations:

- *Problem:* The camera would sometimes flicker between predictions, causing the sentence builder to add random words.
- *Solution:* We implemented a MotionDetector that tracks the velocity of the wrist. The system now "locks" and only registers a word when the hand comes to a complete, intentional stop.

2. Grammar Mismatches:

- *Problem:* The gesture for "Buy/Want" and "When" were both generically categorized as "actions", causing nonsensical sentences like "I ticket when is".
 - *Solution:* We separated the NLP tags into strict sub-categories (action_buy, query_when, query_price) ensuring words only slot into grammatically correct sentence templates.
-

12. Future Scope

- **Dynamic Word Expansion:** Transitioning from static templates to a Large Language Model (LLM) backend that can generate fluent, conversational Marathi sentences based on a loose bag of signed words.
 - **Two-Handed Gestures:** Upgrading the MediaPipe pipeline to track two hands simultaneously for more complex ISL (Indian Sign Language) vocabulary.
 - **Mobile Deployment:** Converting the TensorFlow model to TensorFlow Lite and deploying the system as an Android application for truly portable accessibility.
-

13. Conclusion

This project successfully bridges the communication gap for deaf and hard-of-hearing individuals by translating Marathi Sign Language into spoken Marathi in real-time. By combining robust geometric normalization, deep learning classification, and intelligent context-aware NLP, the system proves that highly accurate, real-time sign language translation can run efficiently on standard consumer hardware.